UNSTITCHED · OneClickDesigner

Full-Time Developer

Hiring Task

Backend engine + AI prompt design

No external accounts needed. Everything runs locally.

Deadline: Friday, 9th May, 12:00 PM IST | hello@theoneclickdesigner.com

About OneClickDesigner

OCD is a live product at theoneclickdesigner.com — a video template marketplace with thousands of users and renders already running. Phase 2 adds AI-powered templates. A user uploads one product image, picks a few options, and gets back commercial-quality photos and videos. Zero design skill required from them.

The AI engine works like n8n or Zapier — every template is a JSON workflow config stored in the database. One generic engine reads it and executes it. No code per template, ever. Your job is to build the core of that engine.

> **Submit: hello@theoneclickdesigner.com**
> Subject: OCD Hire Task — [Your Full Name]
> Deadline: Friday, 9th May, 12:00 PM IST. No extensions.
> Send: GitHub repo link (Part 1) + PDF or Notion link (Part 2, if attempted) + README
> Zero external dependencies — clone and run, nothing else needed

PART 1 — BACKEND (Main Task)

Build the AI workflow engine. Runs 100% locally. No API keys. No AWS.

What You Are Building

The workflow engine is the heart of OCD Phase 2. It reads a template's JSON config and executes it — calling AI providers, managing parallelism, saving checkpoints so jobs survive server crashes, and assembling the final outputs. All AI calls are mocked locally — no Freepik, no Runway, no AWS, no API keys. The only requirement is Node.js.

> **Zero external dependencies.**
> Clone → node test/run.js → it runs. That is the only requirement.
> If your submission needs any account, key, or cloud service, something is wrong.

File Structure to Create

```
/engine
  workflowRunner.js     ← orchestrator — you build this
  stepExecutor.js       ← variable resolver + step router — you build this
  /providers
    mock.js             ← local mock, simulates async AI — you build this
    freepik.js          ← documented stub — you build this (see Part 1D)
/test
  run.js                ← integration test — copy verbatim from Part 1E
README.md
package.json            ← no npm dependencies needed
```

PART 1A — stepExecutor.js

Variable resolution + provider routing

Export

```
async function executeStep(stepDef, job) → Promise<{ [outputKey]: value }>
```

Takes one step definition and the current job state. Resolves all variable references, calls the correct provider, returns the output keyed by outputKey.

Variable Reference Syntax — All 4 Must Work

Reference	Resolves to
`{{userUpload}}`	`job.inputFiles[0]`
`{{userInputs.aspectRatio}}`	`job.userInputs['aspectRatio']`
`{{steps.s1.heroShot}}`	`job.stepOutputs['s1']['heroShot']`
`{{steps.s2.shots[1]}}`	`job.stepOutputs['s2']['shots'][1]`

Rules

- Resolve all {{...}} in every value inside stepDef.inputs
- Resolve all {{...}} in stepDef.prompt
- If any reference resolves to undefined — throw an Error with a clear message. Do not pass undefined silently.
- Route to the correct provider using stepDef.provider ('mock' → mock.js, 'freepik' → freepik.js)
- executionType 'single' — one provider call, return { [outputKey]: result }
- executionType 'parallel' — call provider parallelCount times simultaneously with Promise.all, each gets index 0/1/2..., return { [outputKey]: resultsArray }

PART 1B — workflowRunner.js

Tier orchestration + crash recovery

Export

```
async function runWorkflow(job, template) → Promise<job>
```

Executes the full workflow. Returns the completed job object with all outputs populated.

How Tiers Work

executionOrder is an array of arrays. Each inner array is a tier — a group of steps. Tiers run sequentially. Steps within a tier run in parallel.

```
executionOrder: [
  ['s1'],          // Tier 1 — s1 runs alone, nothing else can start yet
  ['s2', 's3'],    // Tier 2 — s2 and s3 run simultaneously
  ['s4'],          // Tier 3 — s4 waits for both s2 and s3 to finish
]
```

Rules

- At start of each tier: set job.currentTier = i+1 (1-based) and job.status = `tier_${i+1}_of_${total}`
- Run all steps in the tier simultaneously with Promise.all — a for loop with await is wrong
- After each tier: merge outputs into job.stepOutputs, call saveCheckpoint(job)
- After all tiers: build job.finalOutputs from outputSchema, set job.status = 'completed', set job.completedAt
- If any step throws: set job.status = 'failed', job.errorMessage = err.message, stop, return job
- Log clearly: tier start, step start/end, checkpoint, completion — console output must be readable

saveCheckpoint — paste exactly, do not change

```
async function saveCheckpoint(job) {
  await new Promise(r => setTimeout(r, 50)); // simulates DynamoDB write
  console.log('[checkpoint] tier', job.currentTier,
    '— stepOutputs keys:', Object.keys(job.stepOutputs));
}
```

Why checkpoints exist — you must explain this in your README:
If the EC2 server crashes mid-job after tier 2 of 4, saving stepOutputs after every tier lets the runner reload the job and resume from tier 3. Without this, the user restarts from zero and we pay for every already-completed AI API call again.

PART 1C — providers/mock.js

Simulates an async AI provider. Zero real API calls.

The Interface — Must Match Exactly

All real providers (Freepik, Runway, Kling) use this exact same function signature. The engine never knows which provider it's calling — it just calls execute() and gets back a URL. Your mock must honour this contract:

```
async function execute({ type, model, prompt, inputs, duration, index }) {
  // Step 1: wait 200ms  (simulates submitting job to AI API)
  // Step 2: wait 300ms three times  (simulates polling for result)
  // Step 3: return: `s3://mock-bucket/${type}-${Date.now()}-${index ?? 0}.jpg`
}
module.exports = { execute };
```

The 200ms + 3×300ms = 1.1 seconds per call. When tier 2 runs s2 (parallel ×3) and s3 (single) at the same time, your console timestamps must show all 4 calls starting within milliseconds of each other — that is proof parallelism is working.

PART 1D — providers/freepik.js

Stub only — no real calls needed

What This Is

We are not asking you to call the real Freepik API — that requires an account. Write the stub that shows you understand the integration pattern. The real implementation happens during onboarding with our credentials.

Your freepik.js must:

- Export the same execute() function signature as mock.js
- Show the correct Freepik API endpoint in a comment: POST https://api.freepik.com/v1/ai/mystic
- Show the correct polling endpoint: GET https://api.freepik.com/v1/ai/mystic/{task_id}
- Show the correct auth header in a comment: x-freepik-api-key
- Implement the full polling loop structure — but use mock delays instead of real HTTP calls
- Show the base64 decode step as a comment: the API returns base64, you decode it, upload to S3, return S3 URL
- Throw a clear error if called without FREEPIK_API_KEY in env (process.env.FREEPIK_API_KEY)

> **This is the most important file for evaluating your real-world thinking.**
> mock.js shows you can code. freepik.js shows you understand async AI APIs, why outputs must be re-uploaded to S3, and what polling looks like in production.

PART 1E — The Buggy Code — Find and Fix 3 Bugs

Paste this as your starting workflowRunner.js

> **Required. Paste this as your starting point. Run it broken first.**
> Fix all 3 bugs. In your README write one paragraph per bug:
> what it is, what symptom it causes, and what the correct fix is.

```
async function runWorkflow(job, template) {
  const { executionOrder, steps, outputSchema } = template.workflow;
  for (let i = 0; i < executionOrder.length; i++) {
    const tier = executionOrder[i];
    job.status = `tier_${i}_of_${executionOrder.length}`;   // BUG 1
    const results = [];
    for (const stepId of tier) {                            // BUG 2
      const stepDef = steps.find(s => s.stepId === stepId);
      const result = await executeStep(stepDef, job);
      results.push(result);
    }
    results.forEach(r => Object.assign(job.stepOutputs, r));
    await saveCheckpoint(job);
  }
  job.finalOutputs = outputSchema.map(o => {
    const [stepId, key] = o.key.split('.');
    return { label: o.label, type: o.type, url: job.stepOutputs[stepId][key] };
  });
  job.status = 'completed';
  job.stepOutputs = {};                                     // BUG 3
  return job;
}
```

#	Location	Symptom	Hint
1	Status string	First tier shows 'tier_0_of_2' not 'tier_1_of_2'	Loop is 0-based, tiers are 1-based
2	Step execution loop	Steps within a tier run sequentially — defeats the engine's entire purpose	for...await is always sequential
3	After completion	stepOutputs wiped to {} — crash recovery broken, finalOutputs can produce broken URLs	Critical data erased at wrong moment

PART 1F — test/run.js

Integration test — copy this verbatim

Copy This Exactly

Do not change field names, values, or structure. We will verify your output against this exact input.

```
const { runWorkflow } = require('../engine/workflowRunner');

const template = {
  templateId: 'test-001',
  workflow: {
    steps: [
      { stepId:'s1', label:'Hero shot', type:'image_generation', provider:'mock',
        executionType:'single',
        inputs:{ ref:'{{userUpload}}', ratio:'{{userInputs.aspectRatio}}' },
        prompt:'Product photo {{userInputs.aspectRatio}} ratio',
        outputKey:'heroShot' },
      { stepId:'s2', label:'Lifestyle shots', type:'image_generation',
provider:'mock',
        executionType:'parallel', parallelCount:3,
        inputs:{ ref:'{{userUpload}}' },
        prompt:'{{userInputs.modelGender}} model with product',
        outputKey:'lifestyleShots' },
      { stepId:'s3', label:'Product video', type:'image_to_video',
provider:'mock',
        executionType:'single',
        inputs:{ src:'{{steps.s1.heroShot}}' },
        prompt:'Cinematic 360 rotation', duration:5,
        outputKey:'productVideo' },
    ],
    executionOrder:[ ['s1'], ['s2','s3'] ],
    outputSchema:[
      { key:'s1.heroShot',       type:'image',   label:'Hero Shot' },
      { key:'s2.lifestyleShots', type:'image[]', label:'Lifestyle Shots' },
      { key:'s3.productVideo',   type:'video',   label:'Product Video' },
    ],
  },
};

const job = {
  jobId:'job-test-001', userId:'user-123', templateId:'test-001',
  status:'queued', currentTier:0, totalTiers:2,
  inputFiles:['s3://user-uploads/user-123/job-001/product.jpg'],
  userInputs:{ aspectRatio:'9:16', modelGender:'Female', modelEthnicity:'South
Asian' },
  stepOutputs:{}, finalOutputs:[],
```

```
  createdAt:new Date().toISOString(), errorMessage:null,
};

runWorkflow(job, template).then(completed => {
  console.log('\n=== FINAL JOB ===');
  console.log(JSON.stringify(completed, null, 2));
});
```

Your console output must show:

- Tier 1: s1 runs alone
- Tier 2: s2 and s3 start within milliseconds of each other (parallel proof)
- s2 returns 3 URLs in an array (parallelCount:3)
- stepOutputs in final job is NOT empty
- finalOutputs has 3 populated items

README Requirements — Part 1

- How to run: exact command (node test/run.js), confirm zero setup needed
- Paste your actual console output from a real run
- Bug 1 — one paragraph: what it is, symptom, fix
- Bug 2 — one paragraph: what it is, symptom, fix — explain why sequential execution breaks the engine at scale
- Bug 3 — one paragraph: what it is, symptom, fix — explain the crash recovery consequence
- Freepik stub — one paragraph: explain what would change in freepik.js to make it a real integration
- Bonus: resume-from-tier — if implemented, explain how it works

Scoring — Part 1

Variable resolution	All 4 reference types work correctly including array index
Parallelism	Tier 2 timestamps prove simultaneous execution
Checkpoint logic	saveCheckpoint called after every tier, not end-only
Bug 1 — fixed + explained	Status is 1-based. README explains why.
Bug 2 — fixed + explained	Promise.all used. README explains sequential vs parallel impact at scale.
Bug 3 — fixed + explained	stepOutputs preserved. README explains crash recovery use case.
Error handling	Failed step stops job, sets status 'failed' with errorMessage
freepik.js stub	Correct endpoints, auth header, polling structure, base64 note, env var check
README quality	Real console output pasted. Bug explanations show production thinking.
Bonus: resume-from-tier	Runner checks stepOutputs before each tier, skips already-completed ones

Maximum: 10 × 2 = 20 points. This part alone determines if we move forward.

PART 2 — AI PROMPT DESIGN (Bonus)

Optional but strongly weighted. Shows you can own the full stack.

Why This Matters

OCD's AI templates live or die by prompt quality. The backend engine you just built is the delivery vehicle. Prompts are the product. A full-time dev at OCD will eventually write both — new provider adapters AND the prompts that run on them.

This part is optional. But if you skip it, you are applying for a backend-only role. If you complete it well, you are applying to own the entire AI templates stack.

Our users are Indian D2C brand owners — skincare labels, fashion startups, food brands. They upload one product image. They never write a prompt. The entire creative direction comes from what we write.

The Template — Ecommerce Photo Pack

Design the prompt set for 6 outputs from a single product upload:

Step	Output	What it should look like
s1	**Hero Shot**	Clean product on white/neutral bg. Studio lighting. Product listing quality.
s2	**Surface Lifestyle**	Product on premium surface — marble, linen, stone. Props. No people.
s3+s4	**Model Shots ×2**	Two model shots, parallel. Model gender + ethnicity from user inputs.
s5	**Feature Callout**	Close-up of the product's key detail. Product fills most of frame.
s6	**Instagram Flat Lay**	Flat lay, artistic. Colour palette from the product's dominantColor.

Variables Available in Every Prompt

Auto-generated by GPT-Vision step s0 (always runs first)

```
{{productDescription}}  e.g. 'matte black cylindrical perfume bottle with gold cap'
{{productCategory}}     e.g. 'fragrance', 'skincare', 'fashion accessory'
{{dominantColor}}       e.g. 'ivory white', 'deep burgundy'
```

From user inputs (define them in your inputSchema)

```
{{userInputs.aspectRatio}}    — 1:1 / 9:16 / 16:9 / 4:5
{{userInputs.modelGender}}    — Female / Male / No preference
{{userInputs.modelEthnicity}} — South Asian / East Asian / Black / White
```

From earlier steps

```
{{steps.s1.heroShot}}  — S3 URL of step s1 output
```

> **Banned words — cannot appear anywhere in any prompt:**
> beautiful · stunning · elegant · gorgeous · luxurious · premium · perfect · aesthetic
> Replace with specific technical language: light direction, surface material, focal length, shadow quality.

What to Deliver

Deliverable 1 — Prompt Set

One prompt per step (s1–s6). Format each as:

```
Step: s1 — Hero Shot
Provider: freepik
executionType: single  (or parallel + parallelCount)

Prompt: [60–100 words, {{variables}} in place]

Negative prompt: [what must not appear]
```

Every prompt must have: lighting direction, surface or background, camera angle, lens or focal length. Use {{productDescription}} — never write 'the product'.

Deliverable 2 — inputSchema JSON

Include userUpload, aspectRatio, modelGender, modelEthnicity. Add at least one extra field. Field names must exactly match {{userInputs.fieldName}} references in your prompts.

Deliverable 3 — executionOrder

Write the executionOrder for all 6 steps. Explain in comments which steps must wait for earlier outputs and which can run in parallel.

Deliverable 4 — The Hard Prompt

Write a prompt for this exact spec:

> **A skincare serum (amber glass dropper bottle) on white Italian marble.**
> Morning light from the left casting a long diagonal shadow to the right.
> Two dried botanicals (lavender or eucalyptus) in the foreground, slightly out of focus.
> Shot at 75 degrees from horizontal — not flat, not straight-on.
> 4:5 format. Product sharp. Botanicals and background softly blurred.
> No hands. No text. No people.

- Cannot use any banned words
- 80 words or fewer — count them
- Test through any image generator (Freepik Mystic / Midjourney / FLUX / DALL-E 3)
- Include screenshot — and if wrong, show your fix + new output
- 3 sentences: explain which technical terms you used and why each was necessary

Scoring — Part 2

Prompt specificity	Technical language: light direction, surface, focal length. No banned words.
Variable usage	{{productDescription}}, {{userInputs.*}}, {{steps.*}} used correctly in every prompt
inputSchema	Field names match prompt references. Extra field adds real value.
executionOrder	Dependencies respected. Comments explain the reasoning clearly.
Hard prompt	Correct shadow direction, correct angle, shallow DOF, no banned words, ≤80 words
Technical explanation	Can explain why each term in the hard prompt was necessary
Indian market fit	South Asian default. Colour references feel right for Indian D2C.

Maximum: 7 × 2 = 14 points bonus.

Overall Evaluation

Scenario	Score	Decision
Part 1 only, strong	16–20 / 20	Interview for backend role
Part 1 + Part 2, both strong	28–34 / 34	Interview for full-stack AI role
Part 1 weak	Below 12 / 20	Pass regardless of Part 2

One absolute rule: node test/run.js must run with zero setup. If it needs any account, key, or configuration, it will not be evaluated.

This is the work you'll be doing every day if you join.
Good luck.
— Vansh & the Unstitched team